

Методичні вказівки до виконання лабораторної роботи 5

Проектування системи класів у JavaScript

підготував:
Скалоцький Роман Андрійович

Тема

Проектування системи класів у JavaScript на прикладі навчального вебпроєкту «Акустична вартя».

Мета роботи

Навчитися переходити від набору окремих об'єктів і функцій до спроектованої системи взаємопов'язаних класів: виділяти сутності предметної області, визначати їх стан і поведінку, розподіляти відповідальності, будувати композицію, аргументовано використовувати наслідування та інтегрувати об'єктну модель у наявний вебпроєкт.

Необхідні попередні знання

Лабораторна спирається на попередні роботи з проєктом «Акустична вартя». Перед початком студент повинен уміти працювати з HTML/CSS/SCSS, базовим JavaScript, масивами та об'єктами, циклами, умовами, функціями, Function Declaration, Function Expression, Arrow Function, а також на базовому рівні використовувати map, filter і find.

Також передбачається знайомство з this, prototype, синтаксисом class, constructor, методами класу, static та extends. Глибоких знань ООП не потрібно: нові рішення в цій роботі потрібно вміти обґрунтувати на рівні відповідальностей класів.

1. Постановка задачі

У попередніх лабораторних роботах «Акустична вартя» зберігала дані про датчики шуму у звичайних об'єктах і обробляла їх окремими функціями. Такий підхід придатний для невеликого прикладу, але зі зростанням кількості операцій легко отримати набір функцій, які залежать від форми об'єкта, дублюють перевірки та змінюють дані без чіткої відповідальності.

У цій роботі потрібно рефакторити JavaScript-частину вже створеного проєкту. Не створюйте новий незалежний сайт. HTML-структура та стилі попередніх робіт залишаються базою; змінюється спосіб організації даних і логіки в JavaScript.

Практична задача: перетворити дані сенсорів на екземпляри класів, організувати їх у систему моніторингу, централізувати бізнес-правила класифікації шуму, виконати пошук/фільтрацію/агрегацію над колекцією об'єктів і відобразити результат у наявному

інтерфейсі «Акустичної варти».

2. Предметна область «Акустична варта»

«Акустична варта» моделює мережу міських датчиків, які фіксують рівень шуму в різних районах. У системі природно виділяються щонайменше три ролі: конкретний датчик, окреме вимірювання та об'єкт, що керує колекцією датчиків. Попередження про шум не обов'язково має ставати окремим класом: на цьому етапі достатньо розглядати його як стан, що обчислюється за правилами.

Сутність	Що моделює Чому може бути класом
----------	----------------------------------

NoiseSensor Фізичний датчик у конкретній точці міста	Має власний стан, ідентичність та поведінку: статус, оновлення рівня, додавання вимірювань.
--	---

Сутність	Що моделює Чому може бути класом
NoiseMeasurement	Одне вимірювання рівня шуму в певний момент часу Має власні дані: значення, час; може перевіряти коректність і форматовувати результат.
MonitoringSystem	Систему, що працює з колекцією датчиків Відповідає за додавання, пошук, фільтрацію та агрегування даних багатьох сенсорів.
MobileNoiseSensor	Мобільний різновид датчика Може бути прикладом наслідування, якщо справді потрібні додаткові властивості/поведінка.

Не створюйте клас заради кожного іменника. Клас District не потрібний лише тому, що в даних є район. Якщо район у поточній версії проєкту — просто рядок, окремий клас лише ускладнить модель. Новий клас виправданий, коли сутність має власний стан, правила або поведінку.

3. Від процедурного коду до об'єктної моделі

Порівняйте два підходи. У процедурному варіанті дані сенсора існують окремо від функції, яка їх трактує. Функція повинна знати, що в об'єкті є властивість noiseLevel і як саме її класифікувати.

```
const sensor = {
  id: "NS-02",
  district: "Франківський",
  noiseLevel: 74,
};

function getSensorStatus(sensor) {
  // функція знає внутрішню структуру переданого об'єкта
  // ...
}
```

Об'єктна модель переносить поведінку ближче до даних, яких вона стосується. Наприклад, саме NoiseSensor може відповідати за визначення свого поточного статусу. Це не означає, що всі функції потрібно перетворити на методи. Функція, яка формує HTML-картку, належить до представлення сторінки й не повинна автоматично ставати методом NoiseSensor.

4. Проектування системи класів

Перед написанням class виконайте короткий етап проектування. Спочатку визначте відповідальності, а лише потім синтаксис. Візьміть за основу 3 обов'язкові класи: NoiseSensor, NoiseMeasurement та MonitoringSystem. Четвертий клас для наслідування вводьте лише після того, як основна модель працює.

4.1. Таблиця відповідальностей

Клас	Що моделює	Основні властивості	Основні методи	З чим взаємодіє
NoiseSensor	Датчик шуму	id, district, location, noiseLevel, measurements	getStatus(), updateNoiseLevel(), addMeasurement()	NoiseMeasurement, MonitoringSystem
NoiseMeasurement	Одне вимірювання	value, measuredAt	isValid(), toString() або інші мінімальні операції	NoiseSensor
MonitoringSystem	Колекцію сенсорів	sensors	addSensor(), findSensorById(), getCriticalSensors(), getAverageNoiseLevel()	NoiseSensor

4.2. Питання до власної моделі

1. Чому ця сутність стала окремим класом?
2. За що відповідає кожен клас?
3. Які задачі не повинні бути відповідальністю цього класу?
4. Який клас зберігає колекцію датчиків?
5. Де у вашій моделі використана композиція?
6. Чи потрібне наслідування саме у вашій версії проєкту? Якщо так — яке відношення «є різновидом» воно моделює?

5. Клас та екземпляр

Клас описує структуру та поведінку майбутніх об'єктів. Він сам по собі не є конкретним сенсором. Екземпляр з'являється після виклику класу через `new`.

```
class NoiseSensor {  
  // опис майбутніх сенсорів  
}  
  
const sensorA = new NoiseSensor(); // конкретний об'єкт
```

Після створення декількох екземплярів кожен із них має власний стан. Наприклад, `sensorA.noiseLevel` і `sensorB.noiseLevel` можуть відрізнятися, хоча обидва об'єкти створені за одним класом.

Перевірте в Console. Створіть два різні екземпляри `NoiseSensor`. Змініть рівень шуму лише в одному з них і переконайтеся, що стан другого об'єкта не змінився.

6. Constructor і стан об'єкта

`constructor` виконується під час `new` і задає початковий стан екземпляра. Значення, що записуються в `this`, стають власними властивостями конкретного об'єкта.

```
class NoiseSensor {  
  constructor(id, district, location, noiseLevel) {  
    this.id = id;  
    this.district = district;  
    this.location = location;  
    this.noiseLevel = noiseLevel;  
    this.measurements = [];  
  }  
}
```

Властивість `this.id` належить не класу `NoiseSensor` загалом, а конкретному створеному екземпляру. Тому два сенсори можуть мати різні `id`, `district` та `noiseLevel`. Масив `measurements` також потрібно створювати в `constructor`, щоб кожен датчик мав окрему історію вимірювань.

6.1. Початкові дані

```
const sensorsData = [  
  {  
    id: "NS-01",  
    district: "Галицький",  
    location: "просп. Свободи",  
    noiseLevel: 58,  
  },  
  {  
    id: "NS-02",  
    district: "Франківський",  
    location: "вул. Наукова",  
    noiseLevel: 74,  
  },  
  {  
    id: "NS-03",  
    district: "Шевченківський",  
    location: "просп. Чорновола",  
    noiseLevel: 83,  
  },  
];
```

Завдання: Перетворіть їх на екземпляри NoiseSensor. Варіант із map допустимий, але спосіб створення екземпляра потрібно обґрунтувати.

7. Методи класів та prototype

Метод, записаний у тілі class, не копіюється як окрема функція в кожний екземпляр. Він розміщується в NoiseSensor.prototype, а екземпляри знаходять його через прототипний ланцюжок.

```
class NoiseSensor {
  getStatus() {
    // повернути normal, warning або critical
  }
}
```

Тому об'єкт може викликати sensor.getStatus(), хоча getStatus не є власною властивістю sensor. Це принципово відрізняється від this.noiseLevel, яке записується в constructor і належить кожному екземпляру окремо.

7.1. Експеримент із двома екземплярами

```
const sensorA = new NoiseSensor(/* ... */);
const sensorB = new NoiseSensor(/* ... */);

console.log(sensorA.getStatus === sensorB.getStatus);
console.log(Object.hasOwn(sensorA, "noiseLevel"));
console.log(Object.hasOwn(sensorA, "getStatus"));
```

Перед запуском запишіть очікуваний результат кожного виразу. Після перевірки поясніть, чому властивість noiseLevel є власною, а getStatus — ні.

8. Проєктування поведінки класів

Поведінка класу — це операції, які природно належать сутності. Для NoiseSensor логічними є визначення статусу, оновлення поточного рівня й додавання вимірювання. Пошук датчика серед усіх сенсорів — це вже відповідальність MonitoringSystem, а створення HTML — відповідальність шару представлення.

Операція	Куди доцільно віднести Чому
Визначити статус конкретного датчика	NoiseSensor.getStatus() Операція залежить від стану одного сенсора.
Знайти сенсор за id	MonitoringSystem.findSensorById () Потрібен доступ до колекції сенсорів.
Порахувати середній шум усієї системи	MonitoringSystem.getAverageNoiseLevel() Агрегація над багатьма об'єктами.
Створити DOM-картку	Окрема функція Це логіка інтерфейсу, а не

	renderSensorCard() поведінка фізичного сенсора.
Перевірити, чи рівень шуму допустимий	static-метод NoiseSensor Перевірка не потребує стану конкретного екземпляра.

9. Бізнес-правило класифікації шуму

У межах цієї лабораторної використовуйте єдине правило класифікації: рівень до 55 dB включно — normal; понад 55 dB і до 70 dB включно — warning; понад 70 dB — critical. Не дублюйте числа 55 і 70 у різних функціях.

```
const NOISE_LIMITS = Object.freeze({
  normalMax: 55,
  warningMax: 70,
});
```

Метод getStatus() повинен звертатися до централізованої конфігурації. Якщо межі зміняться, їх потрібно змінити в одному місці. Це простий приклад відокремлення бізнес-правила від конкретних викликів.

```
getStatus() {
  // TODO
  // 1. використайте NOISE_LIMITS
  // 2. поверніть "normal", "warning" або "critical"
}
```

10. Static-методи

static-метод належить класу, а не конкретному екземпляру. Його доцільно застосовувати, коли операція логічно пов'язана з типом NoiseSensor, але не потребує this конкретного сенсора.

```
class NoiseSensor {
  static isValidNoiseLevel(value) {
    return Number.isFinite(value) && value >= 0 && value <= 200;
  }
}
```

NoiseSensor.isValidNoiseLevel(74); // допустимий виклик

У цьому випадку немає потреби викликати isValidNoiseLevel() через конкретний об'єкт sensor, тому що перевірка рівня шуму не залежить від його властивостей. Тому такий метод логічніше зробити static і викликати через сам клас: NoiseSensor.isValidNoiseLevel(74).

Ще один доречний приклад static-методу — fromObject(data). Він може отримати звичайний об'єкт із даними та створити на його основі новий екземпляр NoiseSensor.

```
static fromObject(data) {
  // TODO: перевірити необхідні поля
  // TODO: повернути new NoiseSensor(...)
}
```

11. Композиція

Композиція — це спосіб побудови програми, за якого один об'єкт містить інші об'єкти та використовує їх для виконання своєї роботи.

У проєкті «Акустична варта» система моніторингу повинна працювати не з окремими випадковими значеннями, а з об'єктами датчиків:

```
MonitoringSystem
|
└─ sensors -> NoiseSensor[]
```

Тобто об'єкт `MonitoringSystem` містить колекцію об'єктів `NoiseSensor`. У розширеному варіанті кожен датчик може також містити історію своїх вимірювань:

```
NoiseSensor
|
└─ measurements -> NoiseMeasurement[]
```

Таким чином, система будується з кількох об'єктів, кожен із яких має свою відповідальність.

11.1. `MonitoringSystem` містить датчики

Створимо клас системи моніторингу:

```
class MonitoringSystem {
  constructor() {
    this.sensors = [];
  }

  addSensor(sensor) {
    if (!(sensor instanceof NoiseSensor)) {
      throw new Error("Можна додавати лише об'єкти NoiseSensor");
    }

    const sensorExists = this.sensors.some(
      item => item.id === sensor.id
    );

    if (sensorExists) {
      throw new Error(`Датчик ${sensor.id} вже існує`);
    }

    this.sensors.push(sensor);
  }

  findSensorById(id) {
    return this.sensors.find(sensor => sensor.id === id);
  }
}
```

Створимо кілька датчиків:

```
const sensor1 = new NoiseSensor("NS-01", "Галицький", 58);
const sensor2 = new NoiseSensor("NS-02", "Франківський", 74);
```

Створимо систему моніторингу:

```
const system = new MonitoringSystem();
```

Тепер передамо створені датчики в систему:

```
system.addSensor(sensor1);
```

```
system.addSensor(sensor2);
```

У результаті:

```
console.log(system.sensors);
system.sensors міститиме:
[
  NoiseSensor { id: "NS-01", ... },
  NoiseSensor { id: "NS-02", ... }
]
```

Тобто `MonitoringSystem` не зберігає лише ID або рівні шуму. Він містить повноцінні об'єкти `NoiseSensor`.

Тепер система може знайти потрібний датчик:

```
const sensor = system.findSensorById("NS-02");
console.log(sensor);
```

11.2. `NoiseSensor` містить вимірювання

Один датчик може отримувати багато вимірювань у різний час.

Наприклад:

```
NS-01
├── 10:00 -> 54 dB
├── 10:05 -> 61 dB
└── 10:10 -> 73 dB
```

Замість зберігання таких даних як випадкових чисел створимо окремий клас:

```
class NoiseMeasurement {
  constructor(value, measuredAt = new Date()) {
    this.value = value;
    this.measuredAt = measuredAt;
  }
}
```

Тепер `NoiseSensor` може містити масив таких об'єктів:

```
class NoiseSensor {
  constructor(id, district, noiseLevel) {
    this.id = id;
    this.district = district;
    this.noiseLevel = noiseLevel;

    this.measurements = [];
  }

  addMeasurement(measurement) {
    if (!(measurement instanceof NoiseMeasurement)) {
      throw new Error(
        "Очікується об'єкт NoiseMeasurement"
      );
    }
  }

  this.measurements.push(measurement);
  this.noiseLevel = measurement.value;
}
```

Створимо датчик:

```
const sensor = new NoiseSensor(  
  "NS-01",  
  "Галицький",  
  58  
);
```

Створимо нове вимірювання:

```
const measurement = new NoiseMeasurement(74);
```

Передамо його датчику:

```
sensor.addMeasurement(measurement);
```

Тепер:

```
console.log(sensor.measurements);
```

міститиме масив об'єктів NoiseMeasurement, а:

```
console.log(sensor.noiseLevel); Поверне: 74
```

12. Наслідування та super()

Наслідування не повинно бути основою архітектури цієї роботи. Воно потрібне лише для демонстрації ситуації, де один тип справді є різновидом іншого. Приклад: MobileNoiseSensor є різновидом NoiseSensor і додає координати або стан батареї.

```
class MobileNoiseSensor extends NoiseSensor {  
  constructor(id, district, location, noiseLevel, batteryLevel) {  
    super(id, district, location, noiseLevel);  
    this.batteryLevel = batteryLevel;  
  }  
  
  needsCharging() {  
    // TODO  
  }  
}
```

super(...) викликає constructor батьківського класу й ініціалізує ту частину стану, яку MobileNoiseSensor успадковує від NoiseSensor. У constructor похідного класу не можна звертатися до this до виклику super().

13. Реалізація системи моніторингу

Після проектування реалізуйте мінімально працездатну систему. Не копіюйте готовий фінальний код: нижче наведено каркас із навмисно пропущеною логікою.

```
class NoiseSensor {  
  constructor(id, district, location, noiseLevel) {  
    // TODO: ініціалізуйте стан  
  }  
  
  getStatus() {  
    // TODO: використайте централізовані межі  
  }  
  
  updateNoiseLevel(value) {  
    // TODO: перевірте значення через static-метод  
    // TODO: оновіть стан  
  }  
}
```

```

}
static isValidNoiseLevel(value) {
// TODO
}
}

class MonitoringSystem {
  constructor() {
    this.sensors = [];
  }
  addSensor(sensor) {
    // TODO
  }
  findSensorById(id) {
    // TODO
  }

  getSensorsByDistrict(district) {
    // TODO
  }
  getCriticalSensors() {
    // TODO
  }
  getAverageNoiseLevel() {
    // TODO
  }
}

```

13.1. Очікувана поведінка

Виклик	Очікувана поведінка
system.addSensor(sensor)	Додає екземпляр NoiseSensor; відхиляє або обробляє дубль id згідно вашого рішення.
system.findSensorById("NS-02") system.getSensorsByDistrict("Галицький") system.getCriticalSensors()	Повертає відповідний об'єкт або передбачуване значення, якщо його немає. Повертає новий масив сенсорів цього району. Повертає сенсори зі статусом critical, не дублюючи умови класифікації.
system.getAverageNoiseLevel()	Повертає середнє значення для непорожньої колекції; поведінка для порожньої колекції має бути явно визначена.
sensor.updateNoiseLevel(value)	Перевіряє коректність значення та лише потім змінює стан.

14. Робота з колекцією об'єктів

Колекція system.sensors містить не plain objects, а екземпляри NoiseSensor. Тому під час filter, find або map можна звертатися не лише до властивостей, а й до поведінки кожного об'єкта.

```

getCriticalSensors() {

```

```

return this.sensors.filter(sensor => {
  // TODO: використайте sensor.getStatus()
});
}

```

Для пошуку використовуйте `find`, для вибірки декількох елементів — `filter`, для перетворення масиву — `map`. Агрегацію середнього значення можна побудувати через `reduce` або зрозумілий цикл. Оберіть один підхід і поясніть його.

14.1. Контрольні приклади для стартових даних

Для NS-01 = 58 dB очікується `warning`; для NS-02 = 74 dB і NS-03 = 83 dB — `critical`. Середній рівень стартових значень становить приблизно 71.67 dB. Використайте ці значення як контроль, але обчислення має виконувати програма, а не бути записаним вручну.

15. Інтеграція із вебсторінкою «Акустична варта»

Після перевірки класів у Console підключіть модель до наявної сторінки. DOM тут є лише способом продемонструвати результат, а не новою основною темою.

1. Перетворіть `sensorsData` на масив екземплярів `NoiseSensor`.
2. Створіть `MonitoringSystem` та додайте до нього сенсори.
3. Отримайте дані через методи системи, а не через дублювання масивів.
4. На основі об'єктів сформуєте картки сенсорів з ID, районом, адресою, рівнем шуму та статусом.
5. Додайте CSS-клас `normal`, `warning` або `critical` відповідно до `sensor.getStatus()`.
6. Переконайтеся, що зміна рівня через метод об'єкта відображається після повторного рендерингу.

```

function renderSensorCard(sensor) {
  const status = sensor.getStatus();

  // TODO: створіть/поверніть DOM-елемент або HTML-фрагмент
  // className має містити status
  // не дублюйте тут правила 55 / 70
}

```

Зверніть увагу: `renderSensorCard` не є методом `NoiseSensor`, тому що створення HTML не описує фізичну поведінку датчика. Такий поділ не дає моделі предметної області змішатися з DOM-кодом.

16. Рефакторинг структури файлів

Якщо попередній JavaScript уже зібрався в один великий `script.js`, розділіть файли за відповідальністю. Для цієї лабораторної достатньо простої структури без складної enterprise-архітектури.

```

js/
├── models/
│   ├── NoiseSensor.js
│   └── NoiseMeasurement.js
├── services/
│   └── MonitoringSystem.js
└── app.js

```

`models` містить класи предметної області, `services` — об'єкт, що координує роботу колекції, `app.js` — ініціалізацію та інтеграцію з вебсторінкою. Якщо `NoiseMeasurement` не

використовується в базовому варіанті, не створюйте порожній файл лише заради структури.

17. Очікуваний результат

1. На сторінці відображається список датчиків шуму.
2. Для кожного датчика показуються:
 - ID;
 - район;
 - адреса;
 - поточний рівень шуму;
 - статус: normal, warning або critical.
3. Дані для відображення беруться з об'єктів класу NoiseSensor.
4. Об'єкти NoiseSensor зберігаються всередині MonitoringSystem.
5. Система дозволяє знайти датчик за ID.
6. Система дозволяє отримати датчики певного району.
7. Система дозволяє отримати список критичних датчиків.
8. Система обчислює середній рівень шуму.
9. Для датчика можна додати нове вимірювання.
10. Після додавання нового вимірювання оновлюється поточний noiseLevel.
11. Статус датчика автоматично змінюється відповідно до нового рівня шуму.
12. Картки датчиків отримують CSS-клас відповідно до статусу:
 - normal;
 - warning;
 - critical.

18. Як запустити сайт для перевірки

Достатньо такої структури:

```
acoustic-watch/  
├── index.html  
├── css/  
│   └── style.css  
└── js/  
    ├── models/  
    │   ├── NoiseSensor.js  
    │   └── NoiseMeasurement.js  
    ├── services/  
    │   └── MonitoringSystem.js  
    └── app.js
```

У `index.html`:

```
<link rel="stylesheet" href="./css/style.css">  
<script type="module" src="./js/app.js"></script>
```

А запуск — найпростіше через **VS Code -> Live Server -> Open with Live Server**. Тоді відкриється щось на кшталт:

```
http://127.0.0.1:5500/
```

І вже там студент бачить результат роботи класів на сторінці «Акустична варта».

19. Завдання для самостійного виконання

Наведена послідовність є обов'язковою частиною лабораторної. Результатом має бути код вашого проєкту, а не окремий файл із демонстраційними прикладами.

1. Знайдіть у попередній версії «Акустичної варті» дані сенсорів і функції, які з ними працюють. Випишіть щонайменше три функції/операції, які плануєте рефакторити.
2. Заповніть таблицю класів: сутність, стан, поведінка, взаємодії. Створіть просту схему зв'язків.
3. Реалізуйте `NoiseSensor` з constructor і щонайменше методами `getStatus()` та `updateNoiseLevel(value)`.
4. Централізуйте межі класифікації шуму. Не дублюйте 55 і 70 у різних місцях.
5. Реалізуйте щонайменше один осмислений static-метод. Поясніть, чому він не потребує конкретного екземпляра.
6. Реалізуйте `MonitoringSystem`, який містить масив `NoiseSensor` та виконує додавання, пошук, фільтрацію і розрахунок середнього.
7. Створіть щонайменше три сенсори з початкових даних і передайте їх до `MonitoringSystem`.
8. Додайте композицію з `NoiseMeasurement` або обґрунтуйте мінімальний базовий варіант і реалізуйте композицію `MonitoringSystem -> NoiseSensor[]`.
9. Продемонструйте `extends/super()` на змістовному підкласі або, якщо викладач дозволив, окремо поясніть, чому у вашій фінальній моделі наслідування не потрібне.
10. Виконайте консольні експерименти з `prototype` та власними властивостями.
11. Інтегруйте систему з наявною сторінкою й відобразіть картки сенсорів за даними екземплярів.
12. Розділіть JavaScript на декілька файлів, якщо поточний `script.js` вже змішує модель, сервіс і DOM-логіку.
13. Перевірте репозиторій GitHub та робочу розгорнуту версію проєкту.

20. Контрольні питання

1. Чим клас відрізняється від конкретного екземпляра цього класу?
2. Чому `this.noiseLevel` є властивістю конкретного сенсора, а `getStatus()` зазвичай не є власною властивістю екземпляра?
3. Де фактично знаходиться метод, оголошений у тілі `class`?
4. Чому два різні екземпляри `NoiseSensor` можуть використовувати одну й ту саму функцію `getStatus`?
5. Коли операцію доцільно зробити static-методом? Наведіть приклад із «Акустичної варті».
6. Чому static-метод не слід викликати через екземпляр, якщо він оголошений на класі?
7. У чому різниця між станом об'єкта та його поведінкою?
8. Чим композиція `MonitoringSystem -> NoiseSensor[]` відрізняється від наслідування?
9. Яке речення допомагає перевірити, чи виправдане `extends`?
10. Для чого потрібен `super()` у constructor похідного класу?
11. Чому `renderSensorCard(sensor)` не обов'язково має бути методом `NoiseSensor`?
12. Які проблеми виникають, якщо дублювати межі 55 і 70 у кількох методах?
13. Що відбудеться з `this`, якщо зберегти `const fn = sensor.getStatus` і викликати `fn()` окремо?
14. Чому один великий

MonitoringSystem, який відповідає і за DOM, і за бізнес-правила, і за дані, є ознакою поганого проєктування?

15. Яку перевагу дає перетворення plain objects із sensorsData на екземпляри NoiseSensor?